

```
import pandas as pd
import random
!pip install faker
from faker import Faker
from datetime import datetime, timedelta
```

```
fake = Faker("en_IN")
```

```
random.seed(42)
```

```
# -----
# CONFIG
# -----
NUM_PROJECTS = 20
```

```
PROJECT_TYPES = [
    "Fixed Price",
    "Time & Material",
    "Internal"
]
```

```
PROJECT_CATEGORIES = [
    "Client",
    "Internal",
    "Strategic"
]
```

```
PROJECT_STATUS = [
    "Planning",
    "In Progress",
    "On Hold",
    "Completed",
    "Cancelled"
]
```

```
PRIORITIES = [
    "Low",
    "Medium",
    "High",
    "Critical"
]
```

```
DELIVERY_STATUS = [
    "On Track",
    "Delayed",
    "Critical"
]
```

```
PHASES = [  
    "Requirement Gathering",  
    "Design",  
    "Development",  
    "Testing",  
    "Deployment",  
    "Support"  
]
```

```
TASK_CATEGORIES = [  
    "Development",  
    "Testing",  
    "Documentation",  
    "Analysis"  
]
```

```
TASK_STATUS = [  
    "To Do",  
    "In Progress",  
    "In Review",  
    "Done",  
    "Blocked"  
]
```

```
RISK_STATUS = [  
    "Open",  
    "Mitigated",  
    "Closed",  
    "Escalated"  
]
```

```
WORK_MODES = [  
    "Remote",  
    "Hybrid",  
    "Onsite"  
]
```

```
# -----
```

```
# REFERENCE TABLES
```

```
# -----
```

```
employee_ids = [f"EMP{i:03d}" for i in range(1, 101)]
```

```
client_ids = [f"CLT{i:03d}" for i in range(1, 41)]
```

```
employee_names = {  
    emp: fake.name()  
    for emp in employee_ids  
}
```

```

# -----
# PROJECT MASTER
# -----
projects = []

for i in range(1, NUM_PROJECTS + 1):

    project_id = f"PRJ{i:03d}"

    client_id = random.choice(client_ids)

    manager_id = random.choice(employee_ids)

    manager_name = employee_names[manager_id]

    start_date = fake.date_between(
        start_date="-2y",
        end_date="-6m"
    )

    end_date = start_date + timedelta(
        days=random.randint(120, 720)
    )

    budget = random.randint(
        1000000,
        50000000
    )

    actual_cost = random.randint(
        500000,
        budget
    )

    revenue_generated = actual_cost + random.randint(
        100000,
        15000000
    )

    profit_margin = round(
        ((revenue_generated - actual_cost)
        / revenue_generated) * 100,
        2
    )

    completion = random.randint(10, 100)

```

```
project = {
    "project_id": project_id,
    "project_name": fake.bs().title(),

    "client_id": client_id,

    "department_id": f"DPT{random.randint(1,4):03d}",

    "project_manager_id": manager_id,
    "project_manager_name": manager_name,

    "project_type": random.choice(
        PROJECT_TYPES
    ),

    "project_category": random.choice(
        PROJECT_CATEGORIES
    ),

    "project_status": random.choice(
        PROJECT_STATUS
    ),

    "priority": random.choice(
        PRIORITIES
    ),

    "project_start_date": start_date,
    "project_end_date": end_date,

    "budget": budget,

    "actual_cost": actual_cost,

    "revenue_generated": revenue_generated,

    "profit_margin": profit_margin,

    "completion_percentage": completion,

    "project_health_score": random.randint(
        40,
        100
    ),

    "project_complexity": random.choice(
        ["Low", "Medium", "High"]
    ),
}
```

```

        "delivery_status": random.choice(
            DELIVERY_STATUS
        ),

        "delay_days": random.randint(
            0,
            90
        )
    }

    projects.append(project)

project_df = pd.DataFrame(projects)

# -----
# TEAM ALLOCATION
# -----
allocation_rows = []

allocation_counter = 1

for project in projects:

    team_size = random.randint(5, 15)

    assigned_employees = random.sample(
        employee_ids,
        team_size
    )

    for emp_id in assigned_employees:

        start_date = fake.date_between(
            start_date="-1y",
            end_date="-6m"
        )

        end_date = start_date + timedelta(
            days=random.randint(60, 365)
        )

        allocation_rows.append({
            "allocation_id": f"ALC{allocation_counter:04d}",

            "project_id": project["project_id"],

            "employee_id": emp_id,

```

"employee_name": employee_names[emp_id],

"employee_designation": random.choice([
 "Developer",
 "QA Engineer",
 "Consultant",
 "Lead",
 "Analyst"
]),

"department": random.choice([
 "AI Native Application Development",
 "Next Gen Application Development",
 "BI & AI Strategies",
 "Client Success"
]),

"project_manager_id":
 project["project_manager_id"],

"project_manager_name":
 project["project_manager_name"],

"reporting_manager_id":
 random.choice(employee_ids),

"reporting_manager_name":
 fake.name(),

"team_id": f"TEAM{random.randint(1,10):03d}",

"team_size": team_size,

"role_on_project": random.choice([
 "Developer",
 "Analyst",
 "QA",
 "Lead",
 "Consultant"
]),

"allocation_start_date": start_date,

"allocation_end_date": end_date,

"allocation_percentage":
 random.randint(50, 100),

```

        "utilization_percentage":
            random.randint(50, 100),

        "billable_rate":
            random.randint(1000, 15000),

        "billable_status":
            random.choice([
                "Billable",
                "Non-Billable"
            ]),

        "cost_to_company":
            random.randint(500000, 2500000),

        "work_mode":
            random.choice(WORK_MODES),

        "employee_status":
            random.choice([
                "Active",
                "Exited"
            ])
    })
})

allocation_counter += 1

allocation_df = pd.DataFrame(allocation_rows)

# -----
# PROJECT PHASES
# -----
phase_rows = []

phase_counter = 1

for project in projects:

    order = 1

    for phase in PHASES:

        start_date = fake.date_between(
            start_date="-1y",
            end_date="-6m"
        )

```

```

end_date = start_date + timedelta(
    days=random.randint(20, 90)
)

phase_rows.append({
    "phase_id": f"PHS{phase_counter:04d}",

    "project_id": project["project_id"],

    "phase_name": phase,

    "phase_start_date": start_date,

    "phase_end_date": end_date,

    "phase_status": random.choice([
        "Not Started",
        "In Progress",
        "Completed"
    ]),

    "order_sequence": order
})

order += 1
phase_counter += 1

phase_df = pd.DataFrame(phase_rows)

# -----
# MILESTONES
# -----
milestone_rows = []

milestone_counter = 1

for _, phase in phase_df.iterrows():

    for i in range(2):

        due_date = phase["phase_end_date"]

        achieved = (
            due_date - timedelta(
                days=random.randint(0, 15)
            )
        )

```



```

milestone_rows.append({
    "milestone_id":
        f"MLS{milestone_counter:04d}",

    "project_id":
        phase["project_id"],

    "phase_id":
        phase["phase_id"],

    "milestone_name":
        f"{phase[phase_name]} Milestone {i+1}",

    "due_date": due_date,

    "achieved_date": achieved,

    "milestone_status":
        random.choice([
            "Pending",
            "Achieved",
            "Delayed",
            "Cancelled"
        ]),

    "milestone_completion_percentage":
        random.randint(10, 100)
})

milestone_counter += 1

milestone_df = pd.DataFrame(milestone_rows)

# -----
# TASKS
# -----
task_rows = []

task_counter = 1

for _, phase in phase_df.iterrows():

    for i in range(5):

        estimated = random.randint(8, 80)

        actual = estimated + random.randint(-5, 20)

```

```

due_date = fake.date_between(
    start_date="-6m",
    end_date="+2m"
)

completed_date = (
    due_date - timedelta(
        days=random.randint(0, 5)
    )
)

emp_id = random.choice(employee_ids)

task_rows.append({
    "task_id": f"TSK{task_counter:05d}",

    "project_id": phase["project_id"],

    "phase_id": phase["phase_id"],

    "assignee_id": emp_id,

    "employee_name":
        employee_names[emp_id],

    "task_title":
        fake.catch_phrase(),

    "task_category":
        random.choice(TASK_CATEGORIES),

    "task_status":
        random.choice(TASK_STATUS),

    "priority":
        random.choice(PRIORITIES),

    "estimated_hours":
        estimated,

    "actual_hours":
        actual,

    "due_date":
        due_date,

    "completed_date":
        completed_date,

```

```

        "rework_flag":
            random.choice([
                "Yes",
                "No"
            ]),

        "blocker_reason":
            random.choice([
                None,
                "Dependency Issue",
                "Client Delay",
                "Environment Issue"
            ])
    })

    task_counter += 1

task_df = pd.DataFrame(task_rows)

# -----
# TIMESHEETS
# -----
timesheet_rows = []

timesheet_counter = 1

for _, task in task_df.iterrows():

    for i in range(random.randint(1, 3)):

        hours = round(
            random.uniform(2, 10),
            1
        )

        timesheet_rows.append({
            "timesheet_id":
                f"TMS{timesheet_counter:05d}",

            "employee_id":
                task["assignee_id"],

            "employee_name":
                task["employee_name"],

            "project_id":
                task["project_id"],

```

```

        "task_id":
            task["task_id"],

        "work_date":
            fake.date_between(
                start_date="-6m",
                end_date="today"
            ),

        "hours_worked":
            hours,

        "overtime_hours":
            round(
                random.uniform(0, 3),
                1
            ),

        "billable_flag":
            random.choice([
                "Yes",
                "No"
            ]),

        "approved_by":
            fake.name(),

        "work_notes":
            fake.sentence()
    })

    timesheet_counter += 1

timesheet_df = pd.DataFrame(timesheet_rows)

# -----
# PROJECT RISKS
# -----
risk_rows = []

risk_counter = 1

for project in projects:

    for i in range(random.randint(2, 5)):

        owner = random.choice(employee_ids)

```

```

risk_rows.append({
    "risk_id": f"RSK{risk_counter:04d}",

    "project_id":
        project["project_id"],

    "risk_description":
        fake.sentence(),

    "probability":
        random.choice([
            "Low",
            "Medium",
            "High"
        ]),

    "impact":
        random.choice([
            "Low",
            "Medium",
            "High"
        ]),

    "risk_severity_score":
        random.randint(1, 100),

    "risk_status":
        random.choice(RISK_STATUS),

    "escalation_level":
        random.choice([
            "L1",
            "L2",
            "L3"
        ]),

    "mitigation_plan":
        fake.sentence(),

    "resolution_timeline":
        random.randint(5, 90),

    "owner_id": owner,

    "owner_name":
        employee_names[owner]
})

```

```

risk_counter += 1

risk_df = pd.DataFrame(risk_rows)

# -----
# PROJECT HISTORY
# -----
history_rows = []

history_counter = 1

for project in projects:

    history_rows.append({
        "historical_id":
            history_counter,

        "project_id":
            project["project_id"],

        "project_name":
            project["project_name"],

        "project_status":
            project["project_status"],

        "budget":
            project["budget"],

        "actual_cost":
            project["actual_cost"],

        "revenue_generated":
            project["revenue_generated"],

        "profit_margin":
            project["profit_margin"],

        "priority":
            project["priority"],

        "project_manager_id":
            project["project_manager_id"],

        "project_manager_name":
            project["project_manager_name"],

```

```

        "end_date":
            project["project_end_date"],

        "valid_from":
            fake.date_between(
                start_date="-2y",
                end_date="-1y"
            ),

        "valid_to":
            fake.date_between(
                start_date="-1y",
                end_date="today"
            ),

        "current_flag":
            random.choice([
                "Yes",
                "No"
            ]),

        "change_reason":
            random.choice([
                "Scope Change",
                "Budget Revision",
                "Timeline Shift",
                "Re-prioritized"
            ]),

        "record_created_at":
            datetime.now()
    })

    history_counter += 1

history_df = pd.DataFrame(history_rows)

# -----
# PROJECT PERFORMANCE
# -----
performance_rows = []

for project in projects:

    performance_rows.append({
        "project_id":
            project["project_id"],

```

```

        "planned_budget_variance":
            random.randint(-500000, 500000),

        "timeline_variance":
            random.randint(-30, 90),

        "resource_utilization_score":
            random.randint(40, 100),

        "team_productivity_score":
            random.randint(40, 100),

        "project_success_score":
            random.randint(40, 100),

        "milestone_achievement_rate":
            random.randint(50, 100),

        "task_completion_rate":
            random.randint(50, 100),

        "risk_count":
            random.randint(1, 20)
    })

```

```

performance_df = pd.DataFrame(performance_rows)

```

```

# -----
# EXPORT CSV
# -----
project_df.to_csv(
    "project_master.csv",
    index=False
)

allocation_df.to_csv(
    "project_team_allocation.csv",
    index=False
)

phase_df.to_csv(
    "project_phases.csv",
    index=False
)

milestone_df.to_csv(
    "project_milestones.csv",
    index=False
)

```



```
)

task_df.to_csv(
    "project_tasks.csv",
    index=False
)

timesheet_df.to_csv(
    "project_timesheets.csv",
    index=False
)

risk_df.to_csv(
    "project_risks.csv",
    index=False
)

history_df.to_csv(
    "project_history.csv",
    index=False
)

performance_df.to_csv(
    "project_performance.csv",
    index=False
)

print("Project CSV files generated successfully!")
```